

Micro Project

(Course Code: 202044502)

On

Hospital Management

By

Divy R.Shah

Enr.No.: 12402080603002



A Microproject of Programming with Java

Submitted to

Information Technology Department,

A D PATEL IINSTITUTE OF TECHNOLOGY

(A Constituent Collage of CVM University),

New V.V Nagar, Anand - 388121

Internal Faculty

Head of the Department

External Faculty

Index

1) Introduction	2
2) Objectives of the Project	3
3) Modules of the System.....	5
i) Patient Management	5
ii) Appointment Management	5
iii) Doctor Management	5
iv) Prescription Management	5
v) Billing Management	5
vi) File Management	5
4) System Design	6
i) Class Diagram	6
ii) Flow Chart	6
5) Implementation Detail.....	7
i) Inheritance	7
ii) Inner Class	7
iii) Exception Handling	7
iv) Multithreading	8
v) Collection Framework	8
vi) File Handling	8
6) Code Snippets	9
7) Outputs Screenshots.....	13
8) Limitations & Future Enhancements	16
9) Conclusion	18
10) References	19

1. Introduction

The Hospital Management System is a Java-based application designed to manage various hospital activities such as maintaining patient records, managing appointments, storing prescriptions, and generating bills. This project utilizes core object-oriented programming concepts and Java features to provide a simple, yet functional management system.

❖ What to do:-

- Objective: Develop a system to manage patient records, appointments, and billing in a hospital.

• Concepts:

- Inheritance: Different user types (Doctor, Patient, Admin) inherit from a base User class.
- Inner Class: Implement an inner class for managing prescriptions within the Patient class.
- Exception Handling: Handle scheduling conflicts and invalid billing information.
- Threading: Manage simultaneous appointment bookings and patient updates.
- Collection API: Use List and Map for managing patients, doctors, and appointments.
- File Handling: Store patient records and billing information in files.

2. Objectives

1) Automate Patient Management:

- Allow easy addition, modification, deletion, and retrieval of patient details such as name, age, gender, disease, and admission status.
- Maintain patient history and keep records for future references.

2) Efficient Appointment Handling:

- Provide functionality to book appointments between patients and available doctors.
- Prevent overlapping appointments and maintain appointment schedules.

3) Doctor Information Management:

- Store and manage doctor details like name, specialization, availability, and years of experience.
- Help assign patients to appropriate doctors based on specialization.

4) Prescription Handling:

- Allow doctors to generate and save prescriptions for patients.
- Maintain a list of medications and prescribed treatments for each patient.

5) Billing System:

- Automatically generate bills based on consultations, medications, and additional services.
- Ensure accuracy in financial calculations and maintain payment records.

6) Implement Java OOP Concepts:

- Use **inheritance**, **polymorphism**, **encapsulation**, and **abstraction** to make the system modular and maintainable.
- Implement **inner classes** to handle related data within parent classes.

7) Use of Java Collection Framework:

- Manage dynamic lists of patients, doctors, and appointments using ArrayList, HashMap, and other collections.

8) Exception Handling:

- Improve system stability by handling input errors and exceptional conditions using try-catch blocks and custom exceptions.

9) File Handling for Data Persistence:

- Store all data in text files using Java I/O to ensure that information is not lost after the program is closed.

10) Multithreading (if applicable):

- Implement background processes like autosaving, data loading, or timed reminders using threads.

11) Console-Based Interface:

- Provide a simple user interface through the command line to allow users to interact with the system.

3. Modules Used

1)Patient Management:

- Allows adding, updating, viewing, and deleting patient records including name, age, gender, and disease details.

2)Appointment Scheduling:

- Helps patients schedule appointments with available doctors and allows admins to view all upcoming appointments.

3)Doctor Management:

- Stores doctor information like specialization, available times, and experience.

4)Prescription Handling:

- Manages and stores prescriptions provided to patients after diagnosis.

5)Billing System:

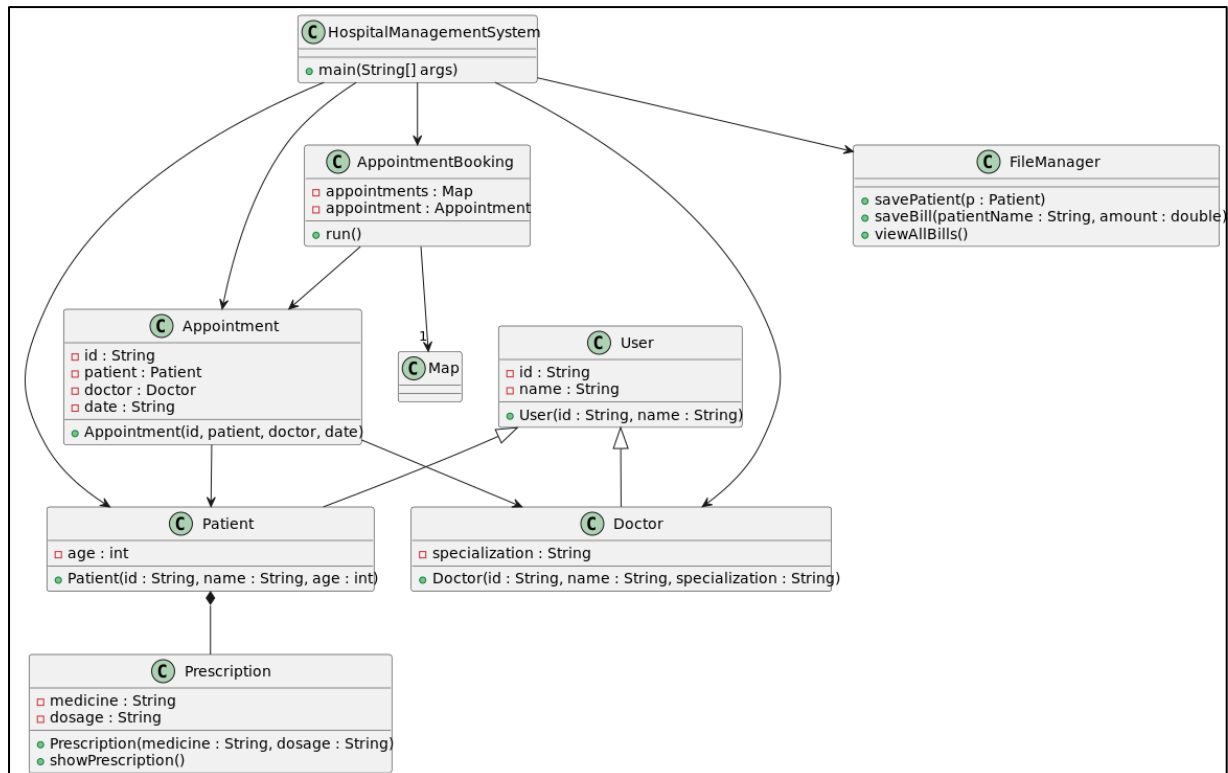
- Generates bills based on consultation fees, medicines, and other charges.

6)File Handling:

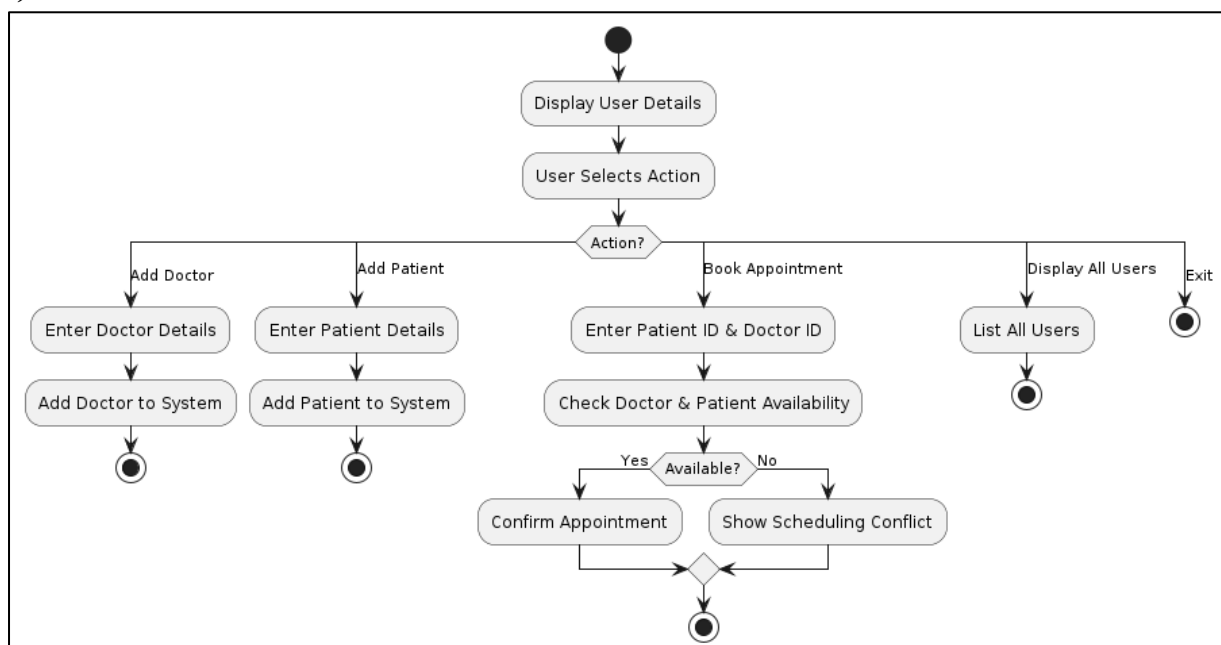
- All records (patients, doctors, bills) are stored using Java File I/O for persistent storage.

4. System Design

i) Class Diagram:



ii) Flow Chart:



5. Implementation Details

Inheritance:

- A base class User is defined with common fields like id and name.
- Doctor and Patient classes extend User, inheriting shared attributes and behavior.
- This promotes code reusability and a clear class hierarchy.

Inner Classes:

- The Patient class contains an **inner class Prescription**, used to manage patient-specific prescriptions.
- This allows Prescription to access Patient's data directly and logically groups related functionalities.

Exception Handling:

- Basic validation is done through try-catch blocks to handle user input errors (e.g., selecting non-existent patient index).
- Custom messages are shown for errors like duplicate appointment IDs or missing billing data.
- Prevents the program from crashing due to invalid operations.

Multithreading:

- Appointment booking is handled using a separate **Thread** (AppointmentBooking) to simulate real-time and concurrent booking processes.
- The run() method adds appointments safely using synchronization to prevent data conflicts.

Collection Framework:

- ArrayList<Patient> and ArrayList<Doctor> are used to store dynamic lists of patients and doctors.

•

- `HashMap<String, Appointment>` is used for fast retrieval and uniqueness of appointments by ID.
- Makes the system scalable and easy to manipulate data structures.

File Handling:

- All patient details are stored in `patients.txt` using `FileWriter` for persistent storage.
- Billing records are stored in `bills.txt` and can be retrieved and displayed using `Scanner`.
- This allows the system to preserve data between runs and acts as a simple database alternative.

6. Code

1. User.java

```
User.java x Doctor.java x Patie
1 public class User {
2     String id;
3     String name;
4
5     public User(String id, String name) {
6         this.id = id;
7         this.name = name;
8     }
9 }
```

2. Doctor.java

```
User.java x Doctor.java x Patient.java
1 public class Doctor extends User {
2     String specialization;
3
4     public Doctor(String id, String name, String specialization) {
5         super(id, name);
6         this.specialization = specialization;
7     }
8 }
9 }
```

3. Patient.java

```
User.java x Doctor.java x Patient.java
1 public class Patient extends User {
2     int age;
3
4     public Patient(String id, String name, int age) {
5         super(id, name);
6         this.age = age;
7     }
8
9     public class Prescription {
10        String medicine;
11        String dosage;
12
13        public Prescription(String medicine, String dosage) {
14            this.medicine = medicine;
15            this.dosage = dosage;
16        }
17
18        public void showPrescription() {
19            System.out.println("Prescription for " + name + ":");
20            System.out.println("Medicine: " + medicine);
21            System.out.println("Dosage: " + dosage);
22        }
23    }
24 }
25 }
```

4. Appointment.java

```
User.java x Doctor.java x Patient.java x Appointment.java
1 public class Appointment {
2     String id;
3     Patient patient;
4     Doctor doctor;
5     String date;
6
7     public Appointment(String id, Patient patient, Doctor doctor, String date) {
8         this.id = id;
9         this.patient = patient;
10        this.doctor = doctor;
11        this.date = date;
12    }
13
14 }
```

5. AppointmentBooking.java

```
1 import java.util.Map;
2
3 public class AppointmentBooking extends Thread {
4     Map<String, Appointment> appointments;
5     Appointment appointment;
6
7     public AppointmentBooking(Map<String, Appointment> appointments, Appointment appointment) {
8         this.appointments = appointments;
9         this.appointment = appointment;
10    }
11
12    public void run() {
13        synchronized (appointments) {
14            if (appointments.containsKey(appointment.id)) {
15                System.out.println("Appointment ID already exists!");
16            } else {
17                appointments.put(appointment.id, appointment);
18                System.out.println("Appointment booked for " + appointment.patient.name +
19                                " with Dr. " + appointment.doctor.name +
20                                " on " + appointment.date);
21            }
22        }
23    }
24 }
25 }
```

6. FileManager.java

```
1 import java.io.*;
2 import java.util.Scanner;
3
4 public class FileManager {
5     public static void savePatient(Patient p) {
6         try {
7             FileWriter fw = new FileWriter("patients.txt", true);
8             fw.write(p.id + ", " + p.name + ", " + p.age + "\n");
9             fw.close();
10        } catch (Exception e) {
11            System.out.println("Error saving patient: " + e.getMessage());
12        }
13    }
14
15    public static void saveBill(String patientName, double amount) {
16        try {
17            FileWriter fw = new FileWriter("bills.txt", true);
18            fw.write(patientName + ", " + amount + "\n");
19            fw.close();
20        } catch (Exception e) {
21            System.out.println("Error saving bill: " + e.getMessage());
22        }
23    }
24
25    public static void viewAllBills() {
26        try {
27            File billFile = new File("bills.txt");
28            if (!billFile.exists() || billFile.length() == 0) {
29                System.out.println("No billing records found.");
30                return;
31            }
32
33            Scanner scanner = new Scanner(billFile);
34            System.out.println("\n== All Bills ==");
35            while (scanner.hasNextLine()) {
36                String line = scanner.nextLine();
37                String[] parts = line.split(",");
38                if (parts.length == 2) {
39                    System.out.println("Patient: " + parts[0] + " | Amount: ₹" + parts[1]);
40                }
41            }
42            scanner.close();
43        } catch (Exception e) {
44            System.out.println("Error reading bill file: " + e.getMessage());
45        }
46    }
47 }
48 }
```

7. HospitalManagementSystem.java(Main File)

```

1  import java.util.*;
2
3  public class HospitalManagementSystem {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          List<Doctor> doctors = new ArrayList<>();
7          List<Patient> patients = new ArrayList<>();
8          Map<String, Appointment> appointments = new HashMap<>();
9
10         while (true) {
11             System.out.println("\n=== Hospital Management System ===");
12             System.out.println("1. Add Patient");
13             System.out.println("2. Add Doctor");
14             System.out.println("3. Book Appointment");
15             System.out.println("4. Generate Prescription & Add Bill");
16             System.out.println("5. View Appointments by Doctor");
17             System.out.println("6. View All Bills");
18             System.out.println("7. Exit");
19             System.out.print("Choose option: ");
20             int ch = sc.nextInt();
21             sc.nextLine();
22
23             switch (ch) {
24                 case 1:
25                     System.out.print("Enter Patient ID: ");
26                     String pid = sc.nextLine();
27                     System.out.print("Enter Name: ");
28                     String pname = sc.nextLine();
29                     System.out.print("Enter Age: ");
30                     int page = sc.nextInt();
31                     sc.nextLine();
32
33                     Patient newPatient = new Patient(pid, pname, page);
34                     patients.add(newPatient);
35                     FileManager.savePatient(newPatient);
36                     System.out.println("Patient added!");
37                     break;
38
39                 case 2:
40                     System.out.print("Enter Doctor ID: ");
41                     String did = sc.nextLine();
42                     System.out.print("Enter Name: ");
43                     String dname = sc.nextLine();
44                     System.out.print("Enter Specialization: ");
45                     String spec = sc.nextLine();
46
47                     Doctor newDoctor = new Doctor(did, dname, spec);
48                     doctors.add(newDoctor);
49                     System.out.println("Doctor added!");
50                     break;
51
52                 case 3:
53                     if (patients.isEmpty() || doctors.isEmpty()) {
54                         System.out.println("Add at least one doctor and patient first.");
55                         break;
56                     }
57
58                     System.out.print("Enter Appointment ID: ");
59                     String aid = sc.nextLine();
60
61                     System.out.println("Available Patients:");
62                     for (int i = 0; i < patients.size(); i++) {
63                         System.out.println(i + ": " + patients.get(i).name);
64                     }
65                     System.out.print("Choose patient index: ");
66                     int pIndex = sc.nextInt();
67                     sc.nextLine();
68
69                     System.out.println("Available Doctors:");
70                     for (int i = 0; i < doctors.size(); i++) {
71                         System.out.println(i + ": " + doctors.get(i).name + " (" + doctors.get(i).specialization + ")");
72                     }
73                     System.out.print("Choose doctor index: ");
74                     int dIndex = sc.nextInt();
75                     sc.nextLine();
76
77                     System.out.print("Enter appointment date (YYYY-MM-DD): ");
78                     String date = sc.nextLine();
79
80                     Appointment appt = new Appointment(aid, patients.get(pIndex), doctors.get(dIndex), date);
81                     AppointmentBooking booking = new AppointmentBooking(appointments, appt);
82                     booking.start();
83                     break;
84
85                 case 4:
86                     if (patients.isEmpty()) {
87                         System.out.println("No patients found.");
88                         break;
89                     }
90
91                     System.out.println("Patients:");
92                     for (int i = 0; i < patients.size(); i++) {
93                         System.out.println(i + ": " + patients.get(i).name);
94                     }
95                     System.out.print("Select patient index: ");
96                     int px = sc.nextInt();
97                     sc.nextLine();
98
99                     System.out.print("Enter medicine name: ");
100                    String med = sc.nextLine();
101                    System.out.print("Enter dosage: ");
102                    String dose = sc.nextLine();
103

```

```
103
104     Patient selected = patients.get(px);
105     Patient.Prescription pres = selected.new Prescription(med, dose);
106     pres.showPrescription();
107
108     System.out.print("Enter billing amount for this prescription: ₹");
109     double amount = sc.nextDouble();
110     sc.nextLine();
111     FileManager.saveBill(selected.name, amount);
112     System.out.println("Bill saved for " + selected.name);
113     break;
114
115     case 5:
116         if (appointments.isEmpty()) {
117             System.out.println("No appointments booked yet.");
118             break;
119         }
120
121         Map<String, List<Appointment>> docWise = new HashMap<>();
122         for (Appointment a : appointments.values()) {
123             docWise.putIfAbsent(a.doctor.name, new ArrayList<>());
124             docWise.get(a.doctor.name).add(a);
125         }
126
127         for (Map.Entry<String, List<Appointment>> entry : docWise.entrySet()) {
128             System.out.println("\nDoctor: " + entry.getKey());
129             for (Appointment a : entry.getValue()) {
130                 System.out.println(" Patient: " + a.patient.name + " | Date: " + a.date + " | ID: " + a.id);
131             }
132         }
133         break;
134
135     case 6:
136         FileManager.viewAllBills();
137         break;
138
139     case 7:
140         System.out.println("Exiting system. Goodbye!");
141         sc.close();
142         return;
143
144     default:
145         System.out.println("Invalid choice!");
146     }
147 }
148 }
149 }
150 }
```


7. Outputs

1. Adding Patient & Doctor

```
D:\ADIT_DESKTOP\SEM - 4th\PWJ\Project\Hospital>javac HospitalManagementSystem.java
D:\ADIT_DESKTOP\SEM - 4th\PWJ\Project\Hospital>java HospitalManagementSystem.java

=== Hospital Management System ===
1. Add Patient
2. Add Doctor
3. Book Appointment
4. Generate Prescription & Add Bill
5. View Appointments by Doctor
6. View All Bills
7. Exit
Choose option: 1
Enter Patient ID: 1
Enter Name: Hbp
Enter Age: 19
? Patient added!
```

```
=== Hospital Management System ===
1. Add Patient
2. Add Doctor
3. Book Appointment
4. Generate Prescription & Add Bill
5. View Appointments by Doctor
6. View All Bills
7. Exit
Choose option: 2
Enter Doctor ID: 1
Enter Name: Drs
Enter Specialization: Alopethy
? Doctor added!
```

2. Make the Appointment

```
=== Hospital Management System ===
1. Add Patient
2. Add Doctor
3. Book Appointment
4. Generate Prescription & Add Bill
5. View Appointments by Doctor
6. View All Bills
7. Exit
Choose option: 3
Enter Appointment ID: 1
Available Patients:
0: Hbp
Choose patient index: 0
Available Doctors:
0: Drs (Alopethy)
Choose doctor index: 0
Enter appointment date (YYYY-MM-DD): 2025-05-01

=== Hospital Management System ===
1. Add Patient
2. Add Doctor
3. Book Appointment
4. Generate Prescription & Add Bill
5. View Appointments by Doctor
6. View All Bills
7. Exit
Choose option: ? Appointment booked for Hbp with Dr. Drs on 2025-05-01
```

3. Viewing the Appointments & Making the Prescription

```
Choose option: ? Appointment booked for Hbp with Dr. Drs on 2025-05-01
5

? Doctor: Drs
  ? Patient: Hbp | Date: 2025-05-01 | ID: 1

=== Hospital Management System ===
1. Add Patient
2. Add Doctor
3. Book Appointment
4. Generate Prescription & Add Bill
5. View Appointments by Doctor
6. View All Bills
7. Exit
Choose option: 4
Patients:
0: Hbp
Select patient index: 0
Enter medicine name: paracetamol
Enter dosage: 2
Prescription for Hbp:
Medicine: paracetamol
Dosage: 2
Enter billing amount for this prescription: ?200
? Bill saved for Hbp
```

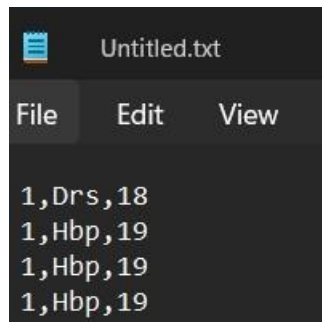
4. Viewing the Bills From the Text file

```
=== Hospital Management System ===
1. Add Patient
2. Add Doctor
3. Book Appointment
4. Generate Prescription & Add Bill
5. View Appointments by Doctor
6. View All Bills
7. Exit
Choose option: 6

=== All Bills ===
? Patient: Hbp | Amount: ?200.0

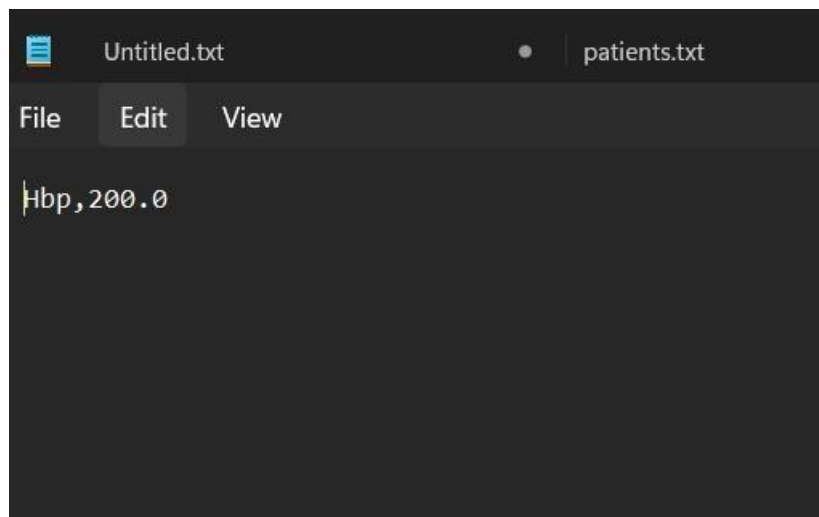
=== Hospital Management System ===
1. Add Patient
2. Add Doctor
3. Book Appointment
4. Generate Prescription & Add Bill
5. View Appointments by Doctor
6. View All Bills
7. Exit
Choose option: 7
? Exiting system. Goodbye!
```

5. Patient.txt



```
1,Drs,18
1,Hbp,19
1,Hbp,19
1,Hbp,19
```

6. Bills.txt



```
Hbp,200.0
```


8. Limitations & Future Enhancements

❖ Limitations:

1. No GUI Interface

- The system uses a command-line interface which may not be user-friendly for hospital staff.

2. No Database Integration

- Data is stored in .txt files, which can lead to data inconsistency, no encryption, and limited scalability.

3. Limited Error Handling

- While basic exception handling is present, it doesn't cover all edge cases or provide detailed feedback.

4. Single Prescription per Patient

- Currently, each patient can have only one prescription at a time.

5. No Login or Authentication System

- All users (patients, doctors, admin) have unrestricted access to the system without security checks.

6. Limited Reporting & Search Functionality

- There is no feature to search for patients/doctors by name or generate detailed reports.

❖ Future Enhancements:

1. Graphical User Interface (GUI)

- Develop a user-friendly GUI using JavaFX or Swing for better usability and navigation.

2. Database Integration

- Replace file handling with a database like MySQL or SQLite for efficient and secure data storage.

3. Advanced Prescription Handling

- Allow multiple prescriptions per patient with support for tracking history and dosage timelines.

4. User Authentication System

- Add login functionality with roles (Admin, Doctor, Patient) and permission-based access.

5. Search & Filter Features

- Implement search, filter, and sort capabilities for appointments, patients, and billing data.

6. Analytics & Reporting

- Generate detailed reports for hospital management (e.g., most visited doctors, billing summaries).

7. Cloud Backup & Auto-Save

- Enable background auto-save and cloud syncing to prevent data loss.

9. Conclusion

The Hospital Management System developed in Java demonstrates the effective use of core object-oriented programming concepts such as inheritance, inner classes, exception handling, multithreading, collections, and file handling. The system provides a functional solution to manage hospital operations including patient records, doctor information, appointment scheduling, prescription management, and billing.

Although the system is currently console-based and uses file storage, it successfully simulates real-world hospital activities in a simplified yet structured manner. It serves as a strong foundation for further expansion into a full-fledged application with GUI, database connectivity, user authentication, and advanced reporting.

This project not only fulfills academic objectives but also enhances practical understanding of Java programming and system design.

10. References

- <https://docs.oracle.com/javase/>
- <https://www.w3schools.com/java/>
- <https://www.geeksforgeeks.org/java/>
- <https://www.javatpoint.com/java-filewriter-class>
- <https://www.javatpoint.com/collections-in-java>
- <https://plantuml.com/class-diagram>
- Head First Java by Kathy Sierra & Bert Bates – for core Java concepts and OOP fundamentals